

AUDIT TITANIUM v12 — Rapport Kimi 3

Branche : reorg/phase1 | Snapshot : 2026-07-29 | Commit de référence : d64e252

1. Vue d'ensemble

Titanium v12 est un bot de trading algorithmique multi-actifs (forex, crypto, indices) tournant sur Windows avec une architecture en pyramide :

- **N0** : Journal unifié + SystemState (socle)
- **N1** : Ingestion de données (MT5, Binance REST/WS, gold)
- **N2** : Détection (SMC, confluence, consensus, lead/lag)
- **N3** : Fusion (Cortex, EventPlane)
- **N4** : Risque (RiskGate, garde-fous)
- **N5** : Exécution (paper trading + démo MT5 fail-closed)

Taille : ~65 000 fichiers, 4,67 Go (dont ~2 Go de modèles ML + DLLs natives) **Langage** : Python 3.11+ (asyncio), FastAPI, MCP stdio, Node.js (tests) **Équipe** : Claude (architecte), Codex (red-team/audit), Hermes (orchestrateur), Cloe (veto local Ollama), Florent (arbitre humain)

2. État des corrections MT5/MCP (commits 3decd8a + d64e252)

✅ CORRIGÉ par Codex

Problème	État	Fichier	Détails
_selected race condition	CORRIGÉ	ingestion/market/mt5_provider.py	_ensure_symbol() sous mt5_lock, vérification retour symbol_select
symbol_select ignoré	CORRIGÉ	mt5_provider.py	MT5SymbolError levée si False
Retry backoff initialize()	CORRIGÉ	mt5_provider.py	_init_failures + _init_next_retry_at + backoff exponentiel
Client HTTP jetable	CORRIGÉ	mcp_server.py	_http_client persistent avec keep-alive
Timeout 4s	CORRIGÉ	mcp_server.py	_HTTP_TIMEOUT = 10.0
Retry réseau	CORRIGÉ	mcp_server.py	_HTTP_MAX_ATTEMPTS = 2 + backoff, distinction timeout/offline/error
Health check boot	CORRIGÉ	mcp_server.py	_health_check() appelé dans _serve()

⚠ PARTIELLEMENT CORRIGÉ

Problème	État	Fichier	Détails
Exceptions explicites	PARTIEL	mt5_provider.py	MT5SymbolError existe mais get_rates/get_ohlcv/get_tick retournent encore None (contrat public Optional conservé pour compatibilité). Recommandation : migrer progressivement les appelants vers try/except
Imports dynamiques cache	NON CORRIGÉ	mcp_server.py	modify_scoring_weight et modify_risk_params font encore sys.path.insert + from engine... à chaque appel. Impact : 100-500ms de latence par appel tool

3. Audit sécurité — Garde-fous

✅ EXCELLENT (A)

Mur démoréel fail-closed (execution/demo_mt5_executor.py)

- assert_demo_or_raise() vérifie trade_mode == DEMO + login == EXPECTED_DEMO_LOGIN + login != REAL_ACCOUNT_LOGIN
- Re-joué juste avant order_send (P0-4) — ferme la fenêtre de course
- DemoExecutionRefused levée fail-closed (toute indisponibilité = refus)

Kill-switch journalier (demo_mt5_executor.py)

- establish_day_ref() : référence d'equity atomique/datée
- Corruption du fichier = refus
- day_start_equity injectable (évite l'incident du 15/07 où un test écrivait la ref de prod)

Sizing fail-closed (demo_mt5_executor.py)

- compute_lot() : arrondi INFÉRIEUR au pas
- _verify_risk() : order_calc_profit obligatoire — absent/None/non-fini = refus
- min_lot_test adaptatif (phase de test Florent)

Garde de spread adaptatif (demo_mt5_executor.py)

- Plafond absolu max_spread_pct (cote cassée)
- Garde relatif max_spread_frac_of_sl (spread vs distance SL) — instrument-agnostique

- Commentaire excellent sur l'incident AUDNZD (spread = 57-650% du risque)

Porte neuronale (core/brain_gate.py + confluence_demo_engine.py)

- entry_gate() = cerveau (consensus) + master (Florent)
- Florent prime toujours (source == "MASTER" exempte des filtres)
- Fail-safe : cerveau illisible = _blocked_gate() = pas de trade

Cloe veto (confluence_demo_engine.py)

- Dernier maillon avant exécution
- Veto uniquement (STOP/DOUTE) — ne peut jamais ouvrir un trade refusé par les moteurs
- Fail-open absolu : lente/éteinte/illisible = la décision déterministe passe
- Timeout interne borné (2s)

⚠ MOYEN (B)

WebSocket temps réel (api/api_server.py)

- Incident du 21/07/2026 : emballement CPU (34 372s cumulées en 8h) par contention mt5_lock
- **Corrigé** : mutualisation _rt_ticks() + cache TTL 0.18s + plafond clients 8 + asyncio.Lock()
- **Reste** : _RT_CLIENTS est un set de WebSocket objects — si un client se déconnecte brutalement sans passer par finally, il reste dans le set jusqu'à expiration. **Recommandation** : heartbeat/ping-pong pour détecter les clients morts

MCP stdio (mcp_server.py)

- Transport stdio = fragile (pipe bloquant, pas de heartbeat natif)
- Health check au boot mais pas de heartbeat périodique
- Si Titanium meurt pendant que MCP tourne, MCP retourne "offline" mais ne redémarre pas
- **Recommandation** : ajouter un watchdog périodique qui vérifie Titanium et loggue sur stderr

Fichiers de log (mcp_server.py)

- read_recent_logs lit titan_stderr.log ou titan_stderr_new.log depuis le répertoire du script
- Pas de rotation de log configurée — les fichiers peuvent grossir indéfiniment
- **Recommandation** : RotatingFileHandler ou monitoring de taille

🔴 À DURCIR (C)

Aucune authentification sur /api/optim/run (api/api_server.py)

- api_optim_run a dependencies=[Depends(require_admin)] 
- Mais api_optim_results (GET) n'a PAS de dépendance admin — lisible par tout le monde
- **Impact** : faible (lecture seule), mais les résultats d'optimisation peuvent contenir des infos sensibles (configurations testées)

CORS middleware (api/api_server.py)

- `allow_origins` inclut `http://localhost:5173, 5174, 8090` — trop permissif si le serveur est exposé au réseau local
- `allow_methods=["GET", "POST"]` — OK
- **Recommandation** : restreindre aux origines strictement nécessaires, ou ajouter un garde si `UVICORN_HOST != "127.0.0.1"`

Port 8090 bindé 127.0.0.1 — OK par défaut, mais `main.py` fait `_kill_zombie_port(UVICORN_PORT, UVICORN_HOST)` qui tue le processus précédent. Si un attaquant local bind le port avant Titanium, il peut intercepter les requêtes MCP.

4. Audit fiabilité des flux de données

MT5 (`ingestion/market/mt5_provider.py`)

Architecture : Verrou `mt5_lock = threading.RLock()` — sérialise tous les appels MT5 entre les boucles `asyncio` concurrentes (`forex/swing/scan/confluence/consensus/leadlag`)

Problème de contention :

- `confluence_demo_engine.run_once()` appelle `rates_fn` pour chaque symbole ET chaque timeframe (LTF, HTF, freshness, refine M5/M1)
- Avec 27 symboles $\times$ ~ 4 timeframes = ~ 108 appels `mt5.copy_rates_from_pos` par cycle
- Chaque appel tient `mt5_lock` pendant ~ 50 -200ms (IO bloquant)
- **Temps de cycle estimé** : $108 \times 100\text{ms} = 10.8\text{s}$ de verrou pur — les autres moteurs (`forex, swing`) sont gelés
- `get_ticks_fast` est aussi sous `mt5_lock` — le flux temps réel (5 Hz) compétitionne avec le scan

Recommandation :

1. **Séparer les verrous** : un verrou pour les lectures (barres) et un autre pour les écritures (ordres) — MT5 est thread-safe en lecture seule
2. **Batcher les requêtes** : `copy_rates_from_pos` pourrait être appelé une seule fois par symbole avec toutes les barres nécessaires, puis découpé en mémoire
3. **Cache de barres** : les barres H1/D1 changent rarement — cache TTL de 30s-60s éliminerait 80% des appels
4. **Pool de workers** : déporter les appels MT5 dans un `ThreadPoolExecutor` dédié avec une queue, au lieu de `asyncio.to_thread` ad hoc

Binance WebSocket (`data/binance_ws.py`)

Architecture : Un WS `aggTrade` par symbole, resamplé en 30s

Problèmes identifiés :

- `_seed_candle_store()` dans `lifespan` resample les bougies 1m en 30s artificiellement — les volumes sont divisés par 2 arbitrairement, ce qui fausse les indicateurs de volume
- Pas de gestion de reconnexion WS documentée — si le WS Binance tombe, les données s'arrêtent

- `candle_store` est un dict global sans verrou — accès concurrent entre le WS writer et les readers (`signal_engine`)

Recommandation :

1. Remplacer le seed artificiel par un vrai fetch 30s/1m historique
2. Ajouter un mécanisme de reconnexion WS avec backoff exponentiel
3. Protéger `candle_store` par `asyncio.Lock()` ou utiliser `asyncio.Queue`

MCP Server (`mcp_server.py`)

Architecture : stdio transport, client HTTP persistant vers FastAPI localhost:8090

Problèmes :

- `_get("/api/state")` est appelé 4× par `read_resource` (signals, positions, market, dashboard) + 1× par `_health_check` = 5× requêtes identiques en <100ms
- **Pas de cache** : l'état global est requêté à chaque resource MCP
- **Impact** : charge inutile sur FastAPI + latence MCP

Recommandation :

1. Ajouter un cache TTL de 1-2s sur `_get("/api/state")` dans `mcp_server.py`
2. Mutualiser les requêtes : `read_resource` pourrait appeler `_get("/api/state")` une seule fois et distribuer les données

5. Audit performance

Démarrage

- `lifespan` crée ~15-20 tâches `asyncio` simultanément
- Chaque moteur (forex, swing, confluence, consensus, leadlag, forward-paper, debrief, knowledge) démarre en parallèle
- **Problème** : `confluence_demo_loop` et `consensus_detection_loop` attendent 30s-50s avant de démarrer — OK, mais pendant ce temps, les autres moteurs consomment déjà les ressources
- `load_state()` est appelé au démarrage — charge potentiellement les modèles .pk1 (925 Mo) en RAM

Mémoire

- 925 Mo de modèles .pk1 + 147 Mo .onnx + 860 Mo de DLLs
- Les modèles sont probablement chargés en RAM au démarrage de `learning_engine` ou `voice_engine`
- **Pas de lazy loading** documenté pour `faster-whisper` et `piper-tts`
- **Recommandation** : charger les modèles ML uniquement quand le module est appelé pour la première fois

CPU

- `confluence_demo_engine.decide()` est déporté dans `asyncio.to_thread` — OK

- Mais `run_once()` fait une boucle `for cfg in symbols_cfg` séquentielle — pas de parallélisme entre symboles
 - Avec 27 symboles, le cycle prend plusieurs secondes
 - **Recommandation** : `asyncio.gather()` pour traiter les symboles en parallèle (avec un semaphore pour limiter la concurrence MT5)
-

6. Audit gestion des erreurs

✅ Bonnes pratiques

- `try/except` avec `noqa: BLE001` partout dans les boucles — fail-safe par symbole
- `logger.warning()` pour les erreurs non fatales, `logger.error()` pour les fatales
- HEARTBEAT dans `confluence_demo_engine` — distingue “aucun signal” de “boucle morte”
- `status_snapshot()` — visibilité complète de l’état

⚠️ Points faibles

Exception swallowing (`confluence_demo_engine.py`)

```
except Exception: # noqa: BLE001
    pass
```

- Trop fréquent — 8 occurrences dans `run_once()`
- Les erreurs sont avalées sans log dans certains cas (`shadow_observer`, `journal_cycle`)
- **Recommandation** : au minimum `logger.debug()` dans chaque `except Exception: pass`

MT5 `last_error()` non propagé (`mt5_provider.py`)

- `get_rates_h1` loggue `mt5.last_error()` mais retourne `None`
- L’appelant ne peut pas distinguer “symbole invalide” de “marché fermé” de “pas d’historique”
- **Recommandation** : enrichir le log avec le contexte (symbole, TF, heure)

MCP “offline” trop vague (`mcp_server.py`)

- `_get()` retourne `{"status": "offline", "reason": str(e)}` pour TOUTE exception non-httpx
 - Une `KeyError` dans le parsing JSON retournerait “offline” — trompeur
 - **Recommandation** : classifier les erreurs par type
-

7. Audit tests

État actuel (snapshot)

- **844 passed, 1 skipped, 26 failed** (baseline connue)
- 57 tests ciblés MT5/MCP passent après les corrections Codex
- 48 tests Cloe passent

Familles d'échecs connues

1. `brain_gate` permissif — attentes historiques
2. `decision_kernel` cache
3. Spread dans l'exécuteur démo
4. Manifeste d'accès GitNexus
5. Prix, sizing et reset du paper trading

⚠ Problèmes de testabilité

Imports dynamiques (`mcp_server.py`)

- `modify_scoring_weight` importe `engine.learning_engine` et `utils.config` à l'intérieur de `call_tool`
- Impossible à mocker proprement dans les tests unitaires
- **Recommandation** : injection de dépendance (passer `scoring_weights` et `SYMBOLS` en paramètres)

État global (`confluence_demo_engine.py`)

- `LAST_DECISION`, `RECENT`, `HEARTBEAT` sont des variables globales
- Les tests doivent les réinitialiser manuellement entre les cas
- **Recommandation** : encapsuler dans une classe `ConfluenceState` injectable

MT5 non disponible en CI — les tests de `mt5_provider` nécessitent MT5 ouvert

- `MT5_AVAILABLE = False` si `ImportError` — mais les tests mockent `mt5` partiellement
- **Recommandation** : fixture `pytest` qui simule complètement l'API MT5

8. Architecture — Points de friction

Monolithisme vs modularité

- `api_server.py` est un fichier de ~800 lignes qui orchestre 15+ moteurs
- `confluence_demo_engine.py` est un fichier de ~500 lignes avec 12 fonctions privées
- **Recommandation** : extraire les boucles de `lifespan` dans des modules dédiés (`engine/loops.py`)

Couplage temporel

- `confluence_demo_loop` attend 30s, `consensus_detection_loop` 40s, `leadlag_loop` 45s
- Ces délais sont des constantes magiques — pas de coordination
- Si un moteur démarre avant que MT5 soit prêt, il échoue silencieusement
- **Recommandation** : un `ReadyGate` qui bloque tous les moteurs jusqu'à ce que MT5 + Binance soient connectés

Double source de vérité

- L'état du système existe à plusieurs endroits :
 - `get_all_signals()` (`signal_manager`)
 - `LAST_DECISION` (`confluence_demo_engine`)
 - `executor.get_state()` (`execution/executor`)
 - `futures_store` (`data/futures_data`)

- `delta_vol` (`data/binance_ws`)
 - Le dashboard `/api/state` agrège tout — mais il n’y a pas de garantie de cohérence
 - **Recommandation** : un `SystemState` unique (déjà partiellement implémenté dans `core/state_builder.py`)
-

9. Recommandations prioritaires

P0 — Critique (à faire avant prochaine session de trading)

1. **Cache d'état MCP** : `mcp_server.py` appelle `/api/state` 5× par requête ressource. Ajouter un cache TTL 1s.
2. **Lazy loading modèles ML** : `faster-whisper` et `piper-tts` chargent 925 Mo au démarrage. Déporter dans un thread séparé ou charger à la demande.
3. **Verrou MT5 par type d'opération** : séparer verrou lecture (barres) et verrou écriture (ordres). Le flux temps réel ne devrait pas bloquer sur les lectures historiques.
4. **Heartbeat WebSocket** : ajouter ping/pong sur `/ws/realtime` pour détecter les clients morts et nettoyer `_RT_CLIENTS`.

P1 — Haute (avant M2 / production)

5. **Imports cache MCP** : remplacer les `sys.path.insert` dans `modify_scoring_weight/modify_risk_params` par `_cached_import()`.
6. **Reconnexion WS Binance** : ajouter un mécanisme de reconnexion avec backoff pour les WS `aggTrade`.
7. **Cache de barres MT5** : les barres H1/D1 changent rarement — cache 30s-60s éliminerait 80% des appels MT5.
8. **Exception swallowing** : ajouter `logger.debug()` dans tous les `except Exception:` pass de `confluence_demo_engine.py`.
9. **Tests unitaires MCP** : les tests actuels mockent `_get` — ajouter des tests pour le client HTTP persistant, le retry, et le health check.

P2 — Moyenne (qualité de vie)

10. **Parallélisme confluence** : `asyncio.gather()` pour traiter les symboles en parallèle dans `run_once()`.
11. **Rotation de logs** : `RotatingFileHandler` pour `titan_stderr.log`.
12. **CORS restrictif** : restreindre `allow_origins` si `UVICORN_HOST != "127.0.0.1"`.
13. **Refactor `api_server.py`** : extraire les 15 boucles `lifespan` dans un module `engine/loops.py`.
14. **ReadyGate** : coordonner le démarrage des moteurs pour attendre MT5 + Binance.

P3 — Basse (dettes techniques)

15. **Encapsuler l'état global** : `LAST_DECISION`, `RECENT`, `HEARTBEAT` → classe injectable.
16. **Injection de dépendance MCP** : passer `scoring_weights` et `SYMBOLS` en paramètres des tools.
17. **Classification d'erreurs MCP** : distinguer `json.JSONDecodeError`, `KeyError`, `ConnectionError` dans `_get()`.
18. **Documentation des 26 tests failed** : documenter précisément pourquoi chaque test échoue et si c'est attendu.

10. Verdict global

Domaine	Note	Commentaire
Sécurité (PAPER ONLY)	A	Mur démoréel, kill-switch, sizing fail-closed, porte neuronale, Cloe veto —excellent
Fiabilité MT5	B+	Corrections Codex solides, mais contention verrou + pas de cache barres
Fiabilité MCP	B+	Client persistant, retry, health check —mais pas de cache état + imports dynamiques
Fiabilité Binance WS	B	Seed artificiel, pas de reconnexion documentée, <code>candle_store</code> non thread-safe
Performance	B	Démarrage lourd (modèles ML), cycle confluence séquentiel, pas de parallélisme symboles
Gestion d'erreurs	B	Fail-safe par symbole, mais exception <code>swallowing</code> excessif
Testabilité	B-	26 tests failed connus, état global, imports dynamiques
Architecture	B	Monolithisme de <code>api_server.py</code> et <code>confluence_demo_engine.py</code> , double source de vérité
Documentation	A	Commentaires exhaustifs, historique des incidents, rationale des décisions
Collaboration	A	Bus append-only, AGENTS.md, ETAT_ACTUEL.md, revues croisées, GitNexus

Verdict : Titanium v12 est un projet **mûr et bien sécurisé** pour du paper trading. Les corrections MT5/MCP de Codex sont solides et résolvent les problèmes de flux vides. Les principaux risques restants sont :

1. **Contention MT5** — le verrou unique peut geler les moteurs concurrents
2. **Latence MCP** — 5× requêtes `/api/state` identiques sans cache
3. **Mémoire** — 925 Mo de modèles ML chargés au démarrage

Aucun de ces risques ne menace la sécurité (PAPER ONLY), mais ils affectent la réactivité et la fiabilité perçue.

Audit réalisé par Kimi 3 / 2026-07-29 Sources : dépôt GitHub [florent1604-lgtm/titanium-v12](#), branche [reorg/phase1](#), commits [3decd8a](#) + [d64e252](#)